1. Problem Title & Source

Title: **Maximum Aqua Curtains in a Box**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Short Summary)

A furnishing company manufactures curtains of two colors:

a → aqua curtain

b → black curtain

The curtain colors are represented as a **string of length N** consisting of characters a and b.

Curtains are packed into boxes such that **each box contains exactly L curtains**.

Your task is to determine **which box contains the maximum number of aqua curtains (a)**.

Finally, return **the maximum number of a characters in any box of size L**.

3. Examples

Example 1

Input

bbbaaababa

3

Output

3

Explanation

Curtains are grouped into substrings of size **3**:

bbb → 0 aqua

bba → 1 aqua

baa → 2 aqua

aaa → 3 aqua ← maximum

aab → 2 aqua

aba → 2 aqua

bab → 1 aqua

aba → 2 aqua

Maximum aqua curtains in a box = **3**

Example 2

Input

abbaaa

2

Output

2

Explanation

Substrings of length **2**

ab → 1

bb → 0

ba → 1

aa → 2 ← maximum

aa → 2

Result = **2**

4. Constraints

1 ≤ L ≤ 10

1 ≤ N ≤ 50

Observations

- Small input size
- Sliding window approach works efficiently

5. Core Concept (Pattern)

Sliding Window

Key idea:

We examine every substring of length L

and count occurrences of 'a'.

6. Thought Process

Brute Force Idea

For every starting position:

count 'a' in substring of size L

Time complexity

O(N × L)

Since N ≤ 50 this works, but we can optimize.

Optimized Idea (Sliding Window)

Steps

1. Count a in first window
2. Slide window by one position
3. Update count

When window moves

remove left element

add right element

This avoids recounting.

Time complexity becomes

O(N)

7. Visualization

Example

String = bbbaaababa

L = 3

Windows

bbb → 0

bba → 1

baa → 2

aaa → 3

aab → 2

aba → 2

bab → 1

aba → 2

Maximum

3

8. Pseudocode

```
read string
read L

count = number of 'a' in first L characters
maxCount = count

for i from L to N-1

   if s[i] == 'a'
      count++

   if s[i-L] == 'a'
      count--
```

```
    maxCount = max(maxCount, count)

print maxCount
```

9. Code Implementation

Python

```
s = input().strip()
L = int(input())

count = 0

for i in range(L):
    if s[i] == 'a':
        count += 1

max_count = count

for i in range(L, len(s)):

    if s[i] == 'a':
        count += 1

    if s[i-L] == 'a':
        count -= 1

    max_count = max(max_count, count)

print(max_count)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);
```

```
        String s = sc.next();
        int L = sc.nextInt();

        int count = 0;

        for(int i=0;i<L;i++){

            if(s.charAt(i) == 'a')
                count++;
        }

        int maxCount = count;

        for(int i=L;i<s.length();i++){

            if(s.charAt(i) == 'a')
                count++;

            if(s.charAt(i-L) == 'a')
                count--;

            maxCount = Math.max(maxCount,count);
        }

        System.out.println(maxCount);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Each character is processed once.

Space Complexity

O(1)

Only counters used.

11. Common Mistakes

1. Recounting window each time
2. Incorrect window boundary
3. Forgetting to subtract left character
4. Using substring unnecessarily

12. Detailed Dry Run

Input

bbbaaababa

L = 3

Window	Substring	Count of a
1	bbb	0
2	bba	1
3	baa	2
4	aaa	3
5	aab	2
6	aba	2
7	bab	1
8	aba	2

Maximum

3

13. Real-Life Applications

Sliding window appears in:

- Network traffic monitoring
- DNA sequence analysis
- Log analysis
- Streaming analytics

Example

Find most frequent event in time window

14. Common Traps

1. Off-by-one errors
2. Window size miscalculation
3. Forgetting first window initialization

4. Incorrect index for outgoing element

15. Related Problems

Sliding window problems:

LC 209 Minimum Size Subarray Sum

LC 3 Longest Substring Without Repeating Characters

LC 424 Longest Repeating Character Replacement

LC 567 Permutation in String

16. Alternate Approaches

Approach	Complexity	Notes
Brute Force	O(N×L)	simple
Sliding Window	O(N)	optimal

17. Why This Solution Works

Each time the window moves:

one element leaves

one element enters

So we update the count in **constant time**, avoiding repeated scanning.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Find substring with **maximum number of 'b'**
2. Longest substring containing **at most K 'a'**
3. Count windows with **exactly K 'a'**
4. Maximum vowels in substring of size K